

Reinforcement Learning for Adaptive Resource Scheduling

Based on Q-Learning in Complex System Environments

Your Name

IIIT Guwahati

March 25, 2026

- **The Challenge:** Modern computing involves massive data volumes and dynamic workloads[cite: 10].
- **Failure of Static Methods:** Traditional algorithms like Round-Robin and Priority Scheduling lack real-time adaptability[cite: 10, 20].
- **Proposed Solution:** A model-free Q-learning algorithm that learns from system state changes to optimize resource allocation[cite: 9, 11].

Methodology: System Modeling

State Space (S_t)

Multidimensional vector: CPU utilization, memory usage, and task queue length[cite: 82, 83].

Action Space (a_t)

Scheduling strategies: Allocating more resources or adjusting task priority[cite: 84].

Reward Function (r_t)

Defined to minimize system strain:

$$r_t = -(CPU_t + Memory_t + QueueLength_t)$$

Ensures higher rewards for lower resource congestion[cite: 94, 96].

Algorithm Stability & Convergence

To improve performance, the study implemented:

- **ϵ -greedy Strategy:** Balances exploration of new strategies and exploitation of known optimal actions[cite: 99, 100].
- **Experience Replay:** Stores transitions (s_t, a_t, r_t, s_{t+1}) in memory to reduce time correlation and stabilize learning[cite: 107, 108].
- **Optimizers:** AdamW performed best due to its weight decay mechanism[cite: 189, 190].

Performance Comparison

Tested on the **Google Cluster Data V2** dataset[cite: 115, 116].

Model	Completion Time (s)	Utilization (%)
Round-Robin	150	65
Priority Scheduling	130	70
DRA	110	75
DRL	95	77
Q-Learning	80	79

Table: Comparison of Scheduling Models [cite: 142]

- Q-learning outperformed both traditional and Deep Reinforcement Learning (DRL) models[cite: 156].
- **Efficiency:** Achieved the shortest task completion time (80s)[cite: 156, 218].
- **Future Scope:** Scalable for edge computing, cloud frameworks, and IoT systems[cite: 15, 39, 41].